pyns Documentation

Release 0.5

Elliott L. Barcikowski

April 03, 2014

CONTENTS

CHAPTER

ONE

PYNS PACKAGE

The pyns package native Python Neuroshare implementation.

1.1 Overview

The classes and functions provided by this interface do not exactly replicate the functions provided by the Neuroshare API. Instead of the functional Neuroshare API generally written in c, the pyns package was created using an object oriented programming style with a focus on having a form natural to the Python language.

This project was put in place by Ripple LLC., after noticing the growing use of the Python language in the academic and scientific communities.

1.2 Required Python Modules

The pyns package makes use of a few Python modules outside the default modules that would be present in any Python installation. These modules are standard for any scientific or computational use of the Python language and are likely to be present on your system.

These modules are:

- numpy - A collection of array manipulation objects and functions.

When data is returned from data in the pyns package, numpy arrays are used to package the data for easy use with matplotlib.

- matplotlib - A collection of plotting and analysis objects and functions.

These are not used explicitly in the pyns package, but they are used in all the provided examples.

- psutil - A module containing process utilities for Windows, Linux, and OS X

This module is used to check the available system memory. Cached data for spike waveforms has the potential to arbitrarily large, and a check is made to ensure enough physical memory is present on a system. However, files large enough to cause problems are unlikely.

On a Windows system the Python distribution Python(x, y) makes installing these modules and other useful analysis packages easy. More information may be found at: http://www.pythonxy.com.

If using Linux platforms, the needed packages depend slightly on the Linux flavor, though they represent the same libraries. On Ubuntu install the following packages: python-psutil, python-numpy, python-matplotlib. They may be installed using apt-get with the following command:

```
$ sudo apt-get install python-psutil python-numpy python-matplotlib
```

If using a Red Hat variant, install the packages python-psutil, python-matplotlib, and numpy. These may be installed using yum with the command:

```
$ sudo yum install python-psutil numpy python-matplotlib
```

1.3 Installation

To install pyns system-wide on your machine either run the command line setup.py. This command should install successfully on any platform. If run in bash, this would look like:

```
$ python setup.py install
```

If using a Windows system, a msi, produced by Python distutils, is also available.

1.4 Neuroshare API to pyns Package

For those familiar the standard Neuroshare API, the following is a mapping of the traditional Neuroshare functions to their pyns equivalent. The arguments and return values are generally not exactly the same due to the object oriented nature of the pyns package. See the docstrings of the classes and functions for more information. Also, some examples of general use of pyns are provided towards the end of this section.

- ns_OpenFile - pyns equivalent: `pyns.NSFile`
- ns_GetFileInfo - pyns equivalent: `pyns.NSFile.get_file_info()`
- ns_GetEntityInfo - pyns equivalent: `pyns.nsentity.Entity.get_entity_info()`
- ns_GetSegmentInfo - pyns equivalent: `pyns.nsentity.SegmentEntity.get_seg_source_info()`
- ns_GetSegmentSourceInfo - pyns equivalent: `pyns.nsentity.SegmentEntity.get_seg_source_info()`
- ns_GetEventInfo - pyns equivalent: `pyns.nsentity.EventEntity.get_event_info()`
- ns_GetAnalogInfo - pyns equivalent: `pyns.nsentity.AnalogEntity.get_analog_info()`
- ns_GetNeuralInfo - pyns equivalent: `pyns.nsentity.NeuralEntity.get_neural_info()`
- ns_GetEventData - pyns equivalent: `pyns.nsentity.EventEntity.get_event_info()`
- ns_GetSegmentData - pyns equivalent: `pyns.nsentity.SegmentEntity.get_segment_data()`
- ns_GetAnalogData - pyns equivalent: `pyns.nsentity.AnalogEntity.get_analog_data()`
- ns_GetEventData - pyns equivalent: `pyns.nsentity.EventEntity.get_event_data()`
- ns_GetTimeByIndex - pyns equivalent: `pyns.nsentity.Entity.get_time_by_index()`
- ns_GetIndexByTime - pyns equivlanent: `pyns.nsentity.Entity.get_index_by_time()`

1.5 Examples

A few examples are provided to ease new users to pyns and Python to this package.

1.5.1 Simple Entity Dump

This example will open a nev file, find the associated .nsx files and print out the result of the pyns.nsentity.Entity.get_entity_info() function for each entity that was found.

```
"""This quick example opens a .nev file named 'test_data.nev'
and prints the ns_EntityInfo struct for each entity.
"""
from pyns.nsfile import NSFile

nsfile = NSFile('test_data.nev')
for entity in nsfile.get_entities():
    print entity.get_info()
```

1.5.2 Spike Plotting

Make use of matplotlib to plot the first electrode channel was found to have recorded spike events. This example makes use of the pyns functions pyns.nsfile.NSFile.get_entities() and pyns.nsentity.SegmentEntity.get_segment_data().

```
"""Example of using matplotlib to plot Neuroshare data, retrieved
with pyns.  Makes use of a fictious nev file 'test_data.nev'.
"""
from matplotlib import pyplot
import numpy
import pyns
nsfile = pyns.NSFile('test_data.nev')
# get segment entities
segment_entities = [e for e in nsfile.get_entities() if e.entity_type == 3]
segment_entity = None
# pick the first segment entity with a non-zero item_count
for entity in segment_entities:
    if entity.item_count > 0:
        segment_entity = entity
        break
    t = numpy.arange(0, 52, dtype=float)/30
# overlay all the spikes for this entity
for item in range(0, segment_entity.item_count):
    (ts, data, unit_id) = segment_entity.get_segment_data(item)
    pyplot.plot(t, data)
pyplot.show()
```

1.5.3 Plotting Continous Channels

Make use of matplotlib to plot the first continous channel. This example makes use of the pyns functions pyns.nsfile.NSFile.get_entities(), pyns.nsentity.AnalogEntity.get_analog_info(), and pyns.nsentity.AnalogEntity.get_analog_data().

```
"""Example of plotting Analog entities (continuous channels) using pyns
and matplotlib.  Makes use of a fictious nev file 'test_data.ns2'.
"""
from matplotlib import pyplot
import numpy
import pyns
```

```
nsfile = pyns.NSFile('test_data.ns2')
# get segment entities
analog_entities = [e for e in nsfile.get_entities() if e.entity_type == 2]
analog_entity = None
# pick the first analog entity with a non-zero item_count
for entity in analog_entities:
    if entity.item_count > 0:
        analog_entity = entity
        break
if analog_entity == None:
    exit(0)
analog_info = analog_entity.get_analog_info()
# only get the first 50k data points (other wise we could be reading
# lots and lots of data in memory at once
data = analog_entity.get_analog_data(0, 50000)
t = numpy.arange(0, len(data), dtype=float)/analog_info.sample_rate
pyplot.plot(t, data)
pyplot.xlabel('[s]')
pyplot.ylabel('[uV]')
pyplot.show()
```

1.6 Revision History

1.6.1 Version 0.5

- Feature: Added support for float based continuous files (.nf3)
- Feature: Added support for stimulation markers returned as segment entities

1.6.2 Version 0.4

- First publicly released pyns version
- Provides a complete version of Neuroshare API

CHAPTER

TWO

PYNS.NSFILE CLASS

class pyns.**NSFile**(*filename*, *proc_single=False*)

General entry point to the pyns implementation of the Neuroshare API. This class loads all the NEV files associated with the file specified. The files are read and all the found entities are stored. This class provides the port of the ns_GetFileInfo function from the Neuroshare API

get_entities(*entity_type=None*)

Return iterator to entity list. If entity_type is specified return only entities of the desired type. entity_type should be one of the members of EntityType

Parameter: entity_type – member from static class nsentity.EntityType,

default=None

Returns: Iterator to entity list containing wanted entities

get_entity(*entity_index*)

Return the entity specified by entity index

get_entity_count()

Utilty function to return the number of entities found in all the files.

get_file_data(*ext*)

Utility function to get the FileData instance with the specified file_type. file_type should be one of nev, ns?

Parameter: ext – file extension

Returns: Associated FileData instance or None if it is not found

get_file_info()

equivalent function to the Neuroshare ns_GetFileInfo function. Returns: FileInfo namedtuple with ns_FILEINFO data

get_time()

return the datetime class corresponding to the origin time found in nev files. This corresponds to the starting time found from the system clock of the DAQ computer.

has_file_type(*file_type*)

Checks to see if NEURALEV, NEURALSG, or NEURALCD type files were found

CHAPTER

THREE

PYNS.NSFILE MODULE

Classes related to handling .nev and .nsx files.

The class below nsfile.NSFile is generally the entry point for the pyns codes.

class pyns.nsfile.**FileData**(*parser*)

Internal data to be used by the File class that is needed to find desired NEV and NSX data. members:

parser – File parser from the pyns.parser module, one of NevParser, Nsx21Parser, Nsx22Parser

time_span – Holds the time span of data for this file in seconds

extension

returns the file extension for this file as stored in the parser class

file_type

Return the file type, one of NEURALEV, NEURALSG, NEURALCD

name

return the full path to this data file. We don't need to store this directly, we may just pass this from the parser object.

class pyns.nsfile.**FileInfo**

FileInfo(file_type, entity_count, timestamp_resolution, time_span, app_name, time_year, time_month, time_day, time_hour, time_min, time_sec, time_millisec, comment)

app_name

Alias for field number 4

comment

Alias for field number 12

entity_count

Alias for field number 1

file_type

Alias for field number 0

time_day

Alias for field number 7

time_hour

Alias for field number 8

time_millisec

Alias for field number 11

time_min

Alias for field number 9

time_month
Alias for field number 6

time_sec
Alias for field number 10

time_span
Alias for field number 3

time_year
Alias for field number 5

timestamp_resolution
Alias for field number 2

class pyns.nsfile.**NSFile**(*filename*, *proc_single=False*)
General entry point to the pyns implementation of the Neuroshare API. This class loads all the NEV files associated with the file specified. The files are read and all the found entities are stored. This class provides the port of the ns_GetFileInfo function from the Neuroshare API

get_entities(*entity_type=None*)
Return iterator to entity list. If entity_type is specified return only entities of the desired type. entity_type should be one of the members of EntityType

Parameter: entity_type – member from static class nsentity.EntityType,

default=None

Returns: Iterator to entity list containing wanted entities

get_entity(*entity_index*)
Return the entity specified by entity index

get_entity_count()
Utilty function to return the number of entities found in all the files.

get_file_data(*ext*)
Utility function to get the FileData instance with the specified file_type. file_type should be one of nev, ns?

Parameter: ext – file extension

Returns: Associated FileData instance or None if it is not found

get_file_info()
equivalent function to the Neuroshare ns_GetFileInfo function. Returns: FileInfo namedtuple with ns_FILEINFO data

get_time()
return the datetime class corresponding to the origin time found in nev files. This corresponds to the starting time found from the system clock of the DAQ computer.

has_file_type(*file_type*)
Checks to see if NEURALEV, NEURALSG, or NEURALCD type files were found

CHAPTER

FOUR

PYNS.NSENTITY MODULE

Collection of classes containing data and functions to hold entity information. These classes facility the reading of nev and nsx (2.1 and 2.2) files to access info functions as well as getting data, waveforms, and timestamps.

All of the Neuroshare API info and data functions are included in the following classes, but these classes extend this and may enable users more flexibility than that API.

class pyns.nsentity.**AnalogEntity**(*parser*, *electrode_id*, *units*, *channel_index*, *scale*, *electrode_label=None*)

Holds data and functions needed for analog entities found in NS[0-9] files. This file stores some additional information found in NSx2.2 style files that include a variety of information about probes in their CC headers. Looking in constructor documentation for more information about these variables.

get_analog_data(*start_index=0*, *index_count=None*, *use_scale=True*)

equivalent of the ns_GetAnalogData. returns the analog data for this entity starting from the start_index bin and returning the next index_count bins. If index_count == None, returns to the end of the analog waveform.

Parameters: start_index - first bin of returned waveform (default: 0) index_count - number of bins in returned waveform (default: until end) use_scale - True: scale to physical units, False: use ADC count.

Default: True

Returns: analog waveform for this entity. Result will length index_count or start_index to the end file.

get_analog_info()

equivalent to the ns_GetAnalogInfo function from the Neuroshare API

get_extended_header()

read the extended header (CC) for this entity and return it.

get_index_by_time(*time*, *flag=0*)

Return the analog index to the segment best matched by time.

Parameters: time - time in seconds from the start of data taking flag - flag to describe how to match files:

-1 - Return the data entry occurring before

or inclusive of time.

0 - Return the data entry occurring at or closest to time.

1 - Return the data entry occurring after or inclusive of time. (default)

Returns: best matched index

get_time_by_index(*index*)
Retrieves time from entity index.

Parameters: index - index for wanted item

Returns: timestamp in seconds from start of data taking

label
Return entity label to be consistent with Neuroshare DLL. In the case of NSx2.1 these labels are included in the CC header. In NSx2.1 we create this label based on the period field.

sample_freq
Return the sample frequency in Hz for this analog channel. This number is the same for all analog channels of the same type and depends on the period and the timestamp_resolution. Generally timestamp_resolution will be the same for all entities but the period (or how much a waveform is decimated will change). All of these things are stored in the parser.

class pyns.nsentity.**AnalogInfo**
AnalogInfo(sample_rate, min_val, max_val, units, resolution, location_x, location_y, location_z, location_user, high_freq_corner, high_freq_order, high_filter_type, low_freq_corner, low_freq_order, low_filter_type, probe_info)

high_filter_type
Alias for field number 11

high_freq_corner
Alias for field number 9

high_freq_order
Alias for field number 10

location_user
Alias for field number 8

location_x
Alias for field number 5

location_y
Alias for field number 6

location_z
Alias for field number 7

low_filter_type
Alias for field number 14

low_freq_corner
Alias for field number 12

low_freq_order
Alias for field number 13

max_val
Alias for field number 2

min_val
Alias for field number 1

probe_info
Alias for field number 15

resolution
Alias for field number 4

sample_rate
Alias for field number 0

units
Alias for field number 3

class pyns.nsentity.**Entity**(*parser*, *electrode_id*)
Base class for Neuroshare Entities. This is an abstract class that holds data and functions common to all neuroshare entities. Actual, instances will of one of the below derived classes (SegmentEntity, AnalogEntity, EventEntity, or NeuralEntity)

add_packet_data(*timestamp*, *packet_index*)
Add packet data to list. The length of this list should be the same as item_count

get_entity_info()
return the entity info for this entity

get_index_by_time(*time*, *flag=0*)
Equivalent to the Neuroshare function ns_GetIndexByTime. Return the segment index to the segment best matched by time. Note: This is overridden by each derived Entity class

Parameters: time - time in seconds from the start of data taking flag - flag to describe how to match files:

-1 - Return the data entry occurring before
or inclusive of time.

0 - Return the data entry occurring at or closest to time.

1 - Return the data entry occurring after or inclusive of time. (default)

Returns: best matched index

get_time_by_index(*index*)
Equivalent to the Neuroshare function ns_GetTimeByIndex. Returns the time in seconds from the start of data taking for the time corresponding to index. Note: This is overridden by each derived Entity class

Parameters: index - index of wanted item

Returns: time in seconds since the start of data taking

label
Return default entity label. This label is used in Segment and Neural entities. This is consistent with both Matlab code as well as Neuroshare DLL.

resize_packet_data()
Reset packet data to the number of items found

class pyns.nsentity.**EntityInfo**
EntityInfo(label, entity_type, item_count)

entity_type
Alias for field number 1

item_count
Alias for field number 2

label
Alias for field number 0

class pyns.nsentity.**EntityType**
static class that will to interface with entity types and provide a couple of utility functions like getting the equivalent strings

classmethod **get_entity_id**(*entity_string*)
Return enum value from entity string, one of "analog", "segment", "neural", "event", or unknown

classmethod **get_entity_string**(*entity_type*)
return string corresponding to entity type

class pyns.nsentity.**EventEntity**(*parser*, *mode*)
Holds data and function for digital event entities found in NEV files.

get_event_data(*packet_index*)
equivalent of the ns_GetEventData from the Nueroshare API

get_event_info()
equivalent of the Neuroshare function ns_GetEventInfo

Returns: EventInfo instance

get_index_by_time(*time*, *flag=0*)
Return the segment index to the segment best matched by time.

Parameters: time - time in seconds from the start of data taking flag - flag to describe how to match files:

-1 - Return the data entry occurring before

or inclusive of time.

0 - Return the data entry occurring at or closest to time.

1 - Return the data entry occurring after or inclusive of time. (default)

Returns: best matched index

get_time_by_index(*index*)
Retrieves time range from entity indexes

Parameters: index - index for wanted item

Returns: timestamp in seconds from start of data taking

label
Return label for EventEntity

class pyns.nsentity.**EventInfo**
EventInfo(event_type, min_data_length, max_data_length, csv_desc)

csv_desc
Alias for field number 3

event_type
Alias for field number 0

max_data_length
Alias for field number 2

min_data_length
Alias for field number 1

class pyns.nsentity.**EventType**
Static class to interface with digital event types

class pyns.nsentity.**NeuralEntity**(*parser*, *electrode_id*, *unit_class*, *segment_entity*)
Entity class for Neural style entities. One of these instances will be created for each class and each segment entity. Storing this data will allow a user to browser through spike waveforms organized by the segment classes.

get_index_by_time(*time*, *flag=0*)
Return index to the segment best matched by time.

Makes use of the held segment entity in the neural entity class. This called is wrapped to segment_entity.get_index_by_time

Parameters: time - time in seconds from the start of data taking flag - flag to describe how to match files:

-1 - Return the data entry occurring before

or inclusive of time.

0 - Return the data entry occurring at or closest to time.

1 - Return the data entry occurring after or inclusive of time. (default)

Returns: best matched index

get_neural_info()
equivalent of the ns_GetNeuralInfo from Neuroshare API

get_time_by_index(*index*)
Retrieves time from entity index.

Makes use of the held segment entity that in the neural entity class. This call is wrapped to segement_entity.get_time_by_index

Parameters: index - index for wanted item

Returns: timestamp in seconds from start of data taking

label
Return label for this Neural Entity by using the contained segment entity

class pyns.nsentity.**NeuralInfo**
NeuralInfo(source_entity_id, source_unit_id, probe_info)

probe_info
Alias for field number 2

source_entity_id
Alias for field number 0

source_unit_id
Alias for field number 1

class pyns.nsentity.**SegSourceInfo**
SegSourceInfo(min_val, max_val, resolution, subsample_shift, location_x, location_y, location_z, location_user, high_freq_corner, high_freq_order, high_filter_type, low_freq_corner, low_freq_order, low_filter_type, probe_info)

high_filter_type
Alias for field number 10

high_freq_corner
Alias for field number 8

high_freq_order
Alias for field number 9

location_user
Alias for field number 7

location_x
Alias for field number 4

location_y
Alias for field number 5

location_z
Alias for field number 6

low_filter_type
Alias for field number 13

low_freq_corner
Alias for field number 11

low_freq_order
Alias for field number 12

max_val
Alias for field number 1

min_val
Alias for field number 0

probe_info
Alias for field number 14

resolution
Alias for field number 2

subsample_shift
Alias for field number 3

class pyns.nsentity.**SegmentEntity**(*parser*, *electrode_id*)
Holds data and information for Segment or spike data found in NEV files. The timestamps of spikes as well as the position of waveforms in a data file are stored in the packet_data list

get_extended_headers()
searches through the NEV file and finds all the extended headers associated with this electrode_id. This method may be slow. If that is the case, I will at the location of each extended header to the SegmentEntity class

get_index_by_time(*time*, *flag=0*)
Equivalent to the Neuroshare function ns_GetIndexByTime. Return the segment index to the segment best matched by time.

Parameters: time - time in seconds from the start of data taking flag - flag to describe how to match files:

-1 - Return the data entry occurring before
or inclusive of time.

0 - Return the data entry occurring at or closest to time.

1 - Return the data entry occurring after or inclusive of time. (default)

Returns: best matched index

get_seg_source_info()
Equivalent to the ns_GetSegSourceInfo from the Neuroshare API. Returns information found in the NEUEVFLT header, an extended header in .nev files.

Returns: SegSourceInfo instance

get_segment_data(*index*)
return the segment waveform corresponding to index for this entity.

This function is the Neuroshare equivalent to the ns_GetSegmentData

Parameters: index - index of wanted item

Returns:

tuple - (timestamp, waveform, unit_id)

timestamp - time of this item in seconds from start of data taking

waveform - waveform found in data packet. The units may be uV (with Neural Data) or V (with stim data)

unit_class - classification of item (0-255)

get_segment_info()
return the segment info struct for this entity

get_time_by_index(*index*)
Equivalent to the Neuroshare function ns_GetTimeByIndex. Returns the time in seconds from the start of data taking for the time corresponding to index.

Parameters: index - index of wanted item

Returns: time in seconds since the start of data taking

class pyns.nsentity.**SegmentInfo**
SegmentInfo(source_count, min_sample_count, max_sample_count, sample_rate, units)

max_sample_count
Alias for field number 2

min_sample_count
Alias for field number 1

sample_rate
Alias for field number 3

source_count
Alias for field number 0

units
Alias for field number 4

CHAPTER

FIVE

PYNS.NSPARSER MODULE

pyns.nsparser - parser header and data for .nev, .nsx2.1 and .nsx2.2 files.

The pyns.nsparser module provides the functionality for all the unlying reading of data for the pyns packet. Any use of the pyns packet will make use of these classes behind the scenes. Advanced uses may find using these classes directly as an easy way to get data out of .nev and .nsx files easily.

The heavy lifting is done with the classes NevParser, Nsx21Parser, and Nsx22Parser. These maybe interfaced with through the factory ParserFactory.

The header formats and sizes are found for all headers and data packets. These are found in the top of the files as ????????_FORMAT and ????????_SIZE constants. namedtuples are provided to return the header and packet data.

class pyns.nsparser.**CC**

CC(header_type, electrode_id, electrode_label, phys_conn, conn_pin, min_dig_value, max_dig_value, min_analog_value, max_analog_value, units, high_freq_corner, high_freq_order, high_filter_type, low_freq_corner, low_freq_order, low_filter_type)

conn_pin
Alias for field number 4

electrode_id
Alias for field number 1

electrode_label
Alias for field number 2

header_type
Alias for field number 0

high_filter_type
Alias for field number 12

high_freq_corner
Alias for field number 10

high_freq_order
Alias for field number 11

low_filter_type
Alias for field number 15

low_freq_corner
Alias for field number 13

low_freq_order
Alias for field number 14

max_analog_value
Alias for field number 8

max_dig_value
Alias for field number 6

min_analog_value
Alias for field number 7

min_dig_value
Alias for field number 5

phys_conn
Alias for field number 3

units
Alias for field number 9

class pyns.nsparser.**DIGLABEL**
DIGLABEL(header_type, label, mode)

header_type
Alias for field number 0

label
Alias for field number 1

mode
Alias for field number 2

class pyns.nsparser.**NEUEVFLT**
NEUEVFLT(header_type, packet_id, high_freq_corner, high_freq_order, high_filter_type, low_freq_corner, low_freq_order, low_filter_type)

header_type
Alias for field number 0

high_filter_type
Alias for field number 4

high_freq_corner
Alias for field number 2

high_freq_order
Alias for field number 3

low_filter_type
Alias for field number 7

low_freq_corner
Alias for field number 5

low_freq_order
Alias for field number 6

packet_id
Alias for field number 1

class pyns.nsparser.**NEUEVLBL**
NEUEVLBL(header_type, packet_id, label)

header_type
Alias for field number 0

label
Alias for field number 2

packet_id
Alias for field number 1

class pyns.nsparser.**NEUEVWAV**
NEUEVWAV(header_type, packet_id, phys_conn, conn_pin, dig_factor, energy_thres, high_thres, low_thres, number_sorted_units, bytes_per_waveform, stim_amp_dig_factor)

bytes_per_waveform
Alias for field number 9

conn_pin
Alias for field number 3

dig_factor
Alias for field number 4

energy_thres
Alias for field number 5

header_type
Alias for field number 0

high_thres
Alias for field number 6

low_thres
Alias for field number 7

number_sorted_units
Alias for field number 8

packet_id
Alias for field number 1

phys_conn
Alias for field number 2

stim_amp_dig_factor
Alias for field number 10

class pyns.nsparser.**NEURALCD**
NEURALCD(header_type, maj_revision, min_revision, bytes_headers, label, comment, period, timestamp_resolution, time_origin, channel_count)

bytes_headers
Alias for field number 3

channel_count
Alias for field number 9

comment
Alias for field number 5

header_type
Alias for field number 0

label
Alias for field number 4

maj_revision
Alias for field number 1

min_revision
Alias for field number 2

period
Alias for field number 6

time_origin
Alias for field number 8

timestamp_resolution
Alias for field number 7

class pyns.nsparser.**NEURALEV**
NEURALEV(header_type, file_rev_major, file_rev_minor, file_flags, bytes_headers, bytes_data_packet, timestamp_resolution, sample_resolution, time_origin, application, comment, n_ext_headers)

application
Alias for field number 9

bytes_data_packet
Alias for field number 5

bytes_headers
Alias for field number 4

comment
Alias for field number 10

file_flags
Alias for field number 3

file_rev_major
Alias for field number 1

file_rev_minor
Alias for field number 2

header_type
Alias for field number 0

n_ext_headers
Alias for field number 11

sample_resolution
Alias for field number 7

time_origin
Alias for field number 8

timestamp_resolution
Alias for field number 6

class pyns.nsparser.**NEURALSG**
NEURALSG(header_type, label, period, channel_count, channel_id)

channel_count
Alias for field number 3

channel_id
Alias for field number 4

header_type
Alias for field number 0

label
Alias for field number 1

period
Alias for field number 2

class pyns.nsparser.**NEVEvent**
NEVEvent(timestamp, packet_id, reason, reserved, digital_input, input1, input2, input3, input4, input5)

digital_input
Alias for field number 4

input1
Alias for field number 5

input2
Alias for field number 6

input3
Alias for field number 7

input4
Alias for field number 8

input5
Alias for field number 9

packet_id
Alias for field number 1

reason
Alias for field number 2

reserved
Alias for field number 3

timestamp
Alias for field number 0

pyns.nsparser.**NEVSegment**
alias of Segment

class pyns.nsparser.**NevParser**(*fid*)
Interface to .nev files. Also for easy reading of nev files and functions to read known basic and extended headers, as well as both segment data packets and event data packets. Member data is held to simply reading of headers and data packets.

file_type
Static functiont to return the 8 byte header associated with this file.

get_basic_header()
Read the NEURALEV header from a nev file. input: fid id of NEURAL returns: struct containing nev header or packet data or None on failure

get_data_packet(*packet_index=None*)
Return the desired data packet. Parameter:

packet_index – index of desired data packet, default=None. if packet_index: seek to absolute path in file and return packet. else: return packet from current point in file.

Returns: NEVEvent or NEVSegment (depending on type of packet) containing data, None on failure.

get_data_packets()
Generator to loop over all data packets. Makes use the get_data_packets function

get_extended_header(*header_index=None*)
Return extended header for nev file from current position in file. This should be modified to be position independent with possibly a generator

get_extended_headers()
Generator to loop over all extended headers. Makes use of the get_extended_header function

get_packet_headers()
Defines an iterator that will return only the timestamps, electrode_id, and unit or reason for spike and digital channels respectively. This function will read in chunks in hopes to more efficiently (in terms of time) open NEV files.

class pyns.nsparser.**Nsx21Parser**(*fid*)
Interface to Nsx2.1 files.

Uses the Python struct module to read all the binary data found in the Nsx21 style files. Holds as member variables a file object and a small amount of header data to allow for easy retrieval.

file_type
Return 8 byte header for this file.

get_analog_data(*channel*, *start_index*, *index_count*)
Return the analog waveform for Nsx2.1 files. Returns data starting at the start_index bin and the next index_count bins. If the end of the file is reached before index_count return a waveform with as many bins as are found. scale provides the conversion from ADC counts to physical numbers. Parameters:

channel - index of the wanted electrode data start_index - first bin of electrode data to return index - how many bins of the waveform to return

get_basic_header()
Return basic header for Nsx2.1 file. Returns NEURALSG instance or None with failure

scale
Returns the scale for Nsx2.1 files. As far as I can tell this is always 1 for NSx2.1

time_span
Return time_span of data in this file. Calculated from number of data points, period, and the clock speed.

timestamp_resolution
Return timestamp_resolution. For Nsx2.1 this quantity is not stored. It will always be 3000.0.

class pyns.nsparser.**Nsx22DataPacket**
Nsx22DataPacket(header, timestamp, n_data_points, data_points)

data_points
Alias for field number 3

header
Alias for field number 0

n_data_points
Alias for field number 2

timestamp
Alias for field number 1

class pyns.nsparser.**Nsx22Parser**(*fid*)
Interface to Nsx2.2 files.

Uses the Python struct module to read all the binary data found in the Nsx21 style files. Holds as member variables a file object and a small amount of header data to allow for easy retrieval.

file_type
Return 8 byte header for this file. Always NEURALCD

get_analog_data(*channel_index*, *start_index*, *index_count*)
Return the analog waveform for Nsx2.2 files. Returns data starting at the start_index bin and the next index_count bins. If the end of the file is reached before index_count return a waveform with as many bins as are found. Parameters:

channel - index of the wanted electrode data start_index - first bin of electrode data to return index - how many bins of the waveform to return

Returns: Requested waveform as a numpy.array

get_analog_packet(*channel_index*, *start_index*, *index_count*)
Generator to to read a file in large chunks but return only the wanted channel and data from a time slice. Parameters:

channel_index - if specified, only return data for the wanted. If unspecified, returns all the data for a single time-slice

packet_index - return the wanted packet. If unspecified, returns as a yield. Otherwise,

get_basic_header()
return the basic NEURALCD file header using the NEURALCD struct defined above.

get_data_packet(*packet_index=None*)
Return one full data packet. This will contain an array of all the digitized data for one moment in time. If packet_index is not specified we will read from the current moment in the file. Otherwise, we skip to the desired packet

get_data_packet_format(*index*)
calculate the number format for the index'th data packet using the number of data points and the channel count

get_extended_header(*header_index=None*)
Get the desired extended (CC) header. If header_index == None, than the next header is read from the current position of the file. If the header_index is >= 0 skip to that position in the file and return that CC header

get_extended_headers()
generator to loop through extended headers. Makes use of the get_extended_header function.

n_data_points
Return the number of data points from each pause section

time_span
Return time_span of data in this file. Calculated from number of data points, period, and the clock speed

pyns.nsparser.**ParserFactory**(*filename*)
ParserFactory provides the interface to the Parser classes listed below and handles opening of and checking the type of the files. Based on the string found in the first few bytes, it returns correct class

CHAPTER

SIX

PYNS.NSEXCEPTIONS MODULE

Collection of Error classes and return values replicating the Neuroshare return values.

Throughout the pyns code, instead of making use of return types, Python exceptions are raised when function fail. The exceptions in pyns classes will always be of the type NeuroshareError.

As often as possible, exceptions are raised with an error number consistent with the Neuroshare API. These are defined in the static class NSReturnTypes

class pyns.nsexceptions.**NSReturnTypes**
Neuroshare standard return values.

exception pyns.nsexceptions.**NeuroshareError**
Exception class to raise Neuroshare errors.

CHAPTER

SEVEN

INDICES AND TABLES

- *genindex*
- *modindex*
- *search*

PYTHON MODULE INDEX

p

INDEX

H

I

L

M

N

P

R

S

T

U